MODULE 26 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Knowledge Bases and retrieval

Retrieve, RetrieveAndGenerate, filtering, reranking and agentic retrieval.

MODULE 26 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

A Knowledge Base is the **entire RAG pipeline**, managed. Ingest, chunk, embed, store, retrieve, generate.

What happens when you ask a question

01

Embed

The query becomes a vector, using the same model as the documents.

02

Search

Vector or hybrid search returns candidate chunks.

03

Filter

Metadata filters narrow to a permitted, relevant subset.

04

Rerank

A second model reorders the candidates for relevance.

05

Generate

The model answers from those chunks, with citations.

ON THE EXAM

Each stage is a separate lever. Module 18's diagnosis tells you which one to reach for.

Data sources, briefly

- S3 is the common source. Managed knowledge bases also connect to **SharePoint, Confluence, Google Drive, OneDrive** and a web crawler.
- Connectors can carry the **source system's permission model**, which matters for the filtering slide.
- An **ingestion job** syncs a data source, with change detection so unchanged files are skipped.
- **Chunking is fixed per data source**, from module 10. Changing it means a new data source.
- Module 10 covered ingestion properly. This is the connection back to it.

> ON THE EXAM
>
> The permission-carrying connectors are the underrated feature: they populate the metadata your tenant filters depend on.

Four operations worth knowing

OPERATION	WHAT IT DOES
`Retrieve`	Returns relevant source chunks. No generation at all
`RetrieveAndGenerate`	GenerateQuery if needed, then Retrieve, then InvokeModel. Answer plus citations
`GenerateQuery`	Turns a natural language question into a query for a structured data store
`AgenticRetrieveStream`	Decomposes a complex query, retrieves iteratively, judges sufficiency

ON THE EXAM

RetrieveAndGenerate is a combined action. Knowing what it calls underneath explains its behaviour and its cost.

Citations

- **RetrieveAndGenerate returns citations to specific source chunks**, not just an answer.
- That is what makes a retrieval system auditable, and it was module 01's giveaway signal.
- If the data source includes **visual elements**, the model can use insights from images, **with source attribution**.
- Citations carry the source location, so a user can open the document and check.
- Module 18 measured whether they are any good: **citation coverage and citation precision**.

> ON THE EXAM
>
> A citation is a claim that the answer came from somewhere. Module 18 exists because that claim needs testing.

Sessions and multi-turn

- RetrieveAndGenerate supports a **session**, so follow-up questions have context.
- That matters because a follow-up like 'and in Germany?' is meaningless retrieved alone.
- The session lets the service reformulate the follow-up into a standalone query before retrieving.
- Longer sessions mean more tokens per turn, exactly as module 16 described for Converse.
- Sessions have a lifetime. They are not permanent conversation storage.

> ON THE EXAM
>
> Retrieval on a bare follow-up question fails badly. The session is what turns 'and in Germany?' into something searchable.

Customising the generation prompt

- You can supply a **custom prompt template** for the generation step.
- That is where you set tone, format, and what to do when the context does not answer the question.
- **Instruct it to say so when the answer is not in the sources.** Otherwise it will improvise.
- Inference parameters from module 16 apply here too. Low temperature for factual answers.
- This is module 16's prompt engineering, applied inside a managed pipeline.

> ON THE EXAM
>
> The single highest-value instruction in a RAG prompt: if the retrieved context does not contain the answer, say so.

Retrieve or RetrieveAndGenerate?

RETRIEVE, WHEN

You want the chunks

- Your application builds its own prompt.
- You need chunks for something other than an answer.
- You want control over the generating model.
- Cheaper: no generation call.

RETRIEVEANDGENERATE, WHEN

You want the answer

- One call gives a natural language response.
- Citations to source chunks come with it.
- Least code to write and maintain.
- It handles structured stores too.

THE TIEBREAKER

Retrieve exists so you can decouple the steps. If you need a custom prompt or a custom model, that is the reason to use it.

Agentic retrieval

- An FM **decomposes a complex query into sub-queries**, and retrieves for each.
- It then **evaluates whether the results are sufficient**, and retrieves again if not.
- Returns **deduplicated source chunks**, so the same passage is not repeated across sub-queries.
- Returns **trace events for observability**, so you can see what it searched for and why.
- It costs several model calls per question, and takes longer. Module 25's agentic capacity point.

> ON THE EXAM
>
> This is module 10's query reformulation, taken further: not just splitting the question, but judging whether the answer was found.

Querying a database through a knowledge base

- A knowledge base can connect to a **structured data store** as well as to documents.
- **GenerateQuery** converts the natural language question into a query the store understands.
- So 'how many orders shipped late last month' becomes a real query, not a retrieval.
- RetrieveAndGenerate calls it automatically when a structured store is connected.
- Module 10's point stands: aggregations belong in SQL, and this is how they get there.

> ON THE EXAM
>
> Retrieval finds passages. Structured queries compute answers. A question asking 'how many' needs the second, not the first.

When a knowledge base is the wrong tool

- **Data that changes by the minute.** Use a tool call, from module 16's scenario 4.
- **Aggregations and comparisons over records.** That is SQL, or GenerateQuery over a structured store.
- **A single document already in hand.** Put it in the prompt; there is nothing to retrieve.
- **Behaviour or format problems.** That is prompting or fine-tuning, from modules 11 and 16.
- Retrieval answers 'what do our documents say'. Nothing else.

> ON THE EXAM
>
> Every module in this domain has a boundary. This one's is that retrieval finds passages, and a passage is not always the answer.

Four settings change what comes back: **search type, filters, result count, reranking.**

Search type, and a constraint that surprises people

- **overrideSearchType** takes **HYBRID** or **SEMANTIC**. By default, Bedrock decides for you.
- Hybrid combines keyword and semantic search, which module 09 said fixes exact identifiers.
- **HYBRID requires an OpenSearch Serverless vector store with a filterable text field.**
- **For other vector store configurations, only SEMANTIC is available.**
- So the vector store you chose in module 09 constrains the retrieval you can configure here.

> ON THE EXAM
>
> This is a genuine cross-module consequence. Choosing S3 Vectors for cost means giving up hybrid search here.

How many chunks to retrieve

- **numberOfResults** sets how many chunks come back. **Minimum 1, maximum 100.**
- **Too few** and the answer is incomplete, which module 18 called low context coverage.
- **Too many** and you fill the context window with marginal passages, and pay for them.
- Every retrieved chunk is **input tokens on every request**, from module 10's budget point.
- Retrieving more and then reranking down is a common and effective pattern.

> ON THE EXAM
>
> The pattern worth knowing: retrieve widely, rerank narrowly. Cast the net with vector search, then let a better model choose.

Metadata filtering

- Supply a **custom metadata file per document**, each up to **10 KB**.
- Filters **pre-filter** the vector store, so you get a defined subset of semantically relevant chunks.
- Filter by department, date, classification, product line, tenant, or anything you captured.
- **This is the mechanism behind tenant isolation** in a multi-tenant knowledge base.
- Metadata you did not capture at ingestion cannot be filtered on. Module 10's warning.

> ON THE EXAM
>
> Semantic similarity does not respect permissions or dates. Filtering is what stops retrieval returning something true but forbidden.

Implicit filtering

- **implicitFilterConfiguration** lets Bedrock apply filters **from the query context** automatically.
- So 'what changed in the 2025 policy' can filter to 2025 without an explicit expression.
- Useful when users phrase constraints naturally rather than through your UI.
- It is inferred, so it can be wrong. Do not rely on it for **security** filtering.
- **Tenant and permission filters should always be explicit**, applied by your application.

> ON THE EXAM
>
> Implicit filtering is a relevance feature. Explicit filtering is a security boundary. Never confuse the two.

Reranking

- A **second, slower, more accurate stage** after the initial vector search.
- A **cross-encoder** looks at the query and each candidate passage **together**, and scores relevance afresh.
- First-stage search uses **one embedding per passage**, which is good at finding and imperfect at ordering.
- So reranking fixes **ordering**, not **recall**. It cannot surface what was never retrieved.
- It adds latency and cost, because every candidate is scored by a model.

> ON THE EXAM
>
> Reranking reorders the candidate set. If the right passage is not in that set, reranking cannot help you.

What the reranking configuration holds

SETTING	WHAT IT DOES
`Reranking model`	Which model performs the reranking
`Additional model request fields`	Model-specific parameters, as in module 16
`Metadata attributes to exclude`	Filter documents out during reranking
`numberOfRerankedResults`	How many results to return after reranking
`The cap`	If it exceeds numberOfResults, you get numberOfResults

ON THE EXAM

That last row is the detail: you cannot rerank your way to more results than the first stage retrieved.

Managed and customer-managed knowledge bases

- Bedrock offers **two distinct knowledge base products**, and they configure differently.
- **Managed** uses **managedSearchConfiguration**. Customer-managed uses **vectorSearchConfiguration**.
- **Managed always uses hybrid search.** Semantic-only is not available there.
- **Managed enables reranking by default**, with a service-managed model.
- Managed also restricts parsing to Smart Parsing, from module 10. Less control, far less configuration.

> ON THE EXAM
>
> Managed makes good decisions for you and removes the ability to make different ones. That is the trade, stated plainly.

Guardrails on a knowledge base

- A **guardrail** can be attached to a RetrieveAndGenerate call, filtering input and output.
- It applies to what the user asked **and** to what the model produced from your documents.
- Your own documents can contain content you would not want quoted back. Retrieval does not know that.
- Guardrails also enforce topic boundaries, so an HR assistant declines legal advice.
- Module 36 covers Guardrails properly. This is the retrieval integration point.

> ON THE EXAM
>
> Grounding an answer in your documents makes it accurate, not appropriate. Those are different properties.

Multi-tenancy in a knowledge base

- **One knowledge base per tenant** is the strongest isolation, and the most to manage.
- **One knowledge base with a tenant metadata filter** is cheaper and relies on your filter being right.
- That filter must be **explicit** and applied by your application, never inferred.
- The application, not the user, supplies the tenant value. A user-supplied filter is not a boundary.
- Module 09's rule again: isolation enforced outside your code survives a bug inside it.

> ON THE EXAM
>
> If the tenant identifier can be influenced by the request, it is a parameter, not a security control.

Security around a knowledge base

- **IAM** controls who may call Retrieve and RetrieveAndGenerate, and on which knowledge base.
- The knowledge base's **service role** needs access to the data source and the vector store.
- Vectors and source documents are encrypted, and a **customer-managed KMS key** can be used.
- **Embeddings are not anonymised**, from module 09. The store inherits the data's sensitivity.
- Retrieval bypasses your application's own access control. Module 10 said so, and it stays true.

> ON THE EXAM
>
> Three layers: who may query, what the service may read, and which chunks come back. All three need setting.

Symptom to lever

WHAT IS WRONG	THE LEVER
Right topic, wrong department or date	Metadata filtering
Users see documents they should not	Explicit metadata filters, applied by your application
Correct passage retrieved, but ranked low	Reranking
Answer incomplete, passages on topic	Raise numberOfResults. Low context coverage
Exact codes and SKUs not matched	Hybrid search, which needs OpenSearch Serverless
Multi-part questions half-answered	Agentic retrieval, or query decomposition
'How many' questions answered vaguely	Connect a structured data store. GenerateQuery
Answer not supported by the sources	A generation problem, not retrieval. Module 18's faithfulness

ON THE EXAM

This is the slide to revise. Module 18 diagnoses; this table prescribes.

GraphRAG

- Combines a **knowledge graph** with vector retrieval, using Amazon Neptune Analytics.
- It answers questions needing **relationships**: who reports to whom, what depends on what.
- Pure vector retrieval finds similar passages and cannot traverse a chain of connections.
- The graph is built during ingestion, so entities and relationships are extracted from your documents.
- More to build and maintain. Reach for it when relationships are genuinely the question.

> ON THE EXAM
>
> 'Which suppliers are affected if this component fails' is a traversal, not a similarity search. That is the GraphRAG signal.

What a knowledge base costs

COMPONENT	COST SHAPE
Ingestion	Embedding tokens, plus parsing if using BDA or an FM parser. Module 10
Vector storage	The store's own pricing. S3 Vectors, OpenSearch, or pgvector. Module 09
Retrieve	Query embedding, plus the vector store's query charge
RetrieveAndGenerate	The above, plus generation tokens including every retrieved chunk
Reranking	A model call scoring every candidate. Scales with numberOfResults
Agentic retrieval	Several model calls and several retrievals per question

ON THE EXAM

Retrieved chunks are input tokens on every single request. numberOfResults is a cost setting as much as a quality one.

Evaluating a knowledge base

- **Bedrock RAG evaluation** from module 18 points directly at a knowledge base.
- **Retrieve-only** jobs isolate the retrieval stage. **Retrieve-and-generate** judges the whole thing.
- Change **one lever** per evaluation run, then compare jobs, so you can attribute the difference.
- Context relevance and context coverage map onto filtering and numberOfResults respectively.
- Without this, tuning a knowledge base is a sequence of opinions.

> ON THE EXAM
>
> Module 18 and module 26 are two halves of one activity: measure the stage, then pull the lever that stage responds to.

Monitoring a knowledge base

SIGNAL	WHAT IT TELLS YOU
Ingestion job status	Whether documents actually made it in. Failures are per-document
Documents indexed and failed	A silently failing subset is common on mixed-format corpora
Retrieve latency	Dominated by the vector store and by reranking
Token counts per request	Retrieved chunks are input tokens. This is where cost lives
Sync recency	How stale the index is, from module 10's deletion risk

ON THE EXAM

Failed documents are the quiet one. A corpus that indexed 94 percent looks fine until somebody asks about the other 6 percent.

Before you build an application

- The console lets you **query a knowledge base directly** and see the retrieved chunks.
- Look at the **chunks**, not just the answer. A good answer from bad chunks is luck.
- Try the questions you expect to be hard before you build anything around it.
- Compare search types and result counts interactively, then configure once.
- Same advice as module 16's playground, applied to retrieval.

> ON THE EXAM
>
> Reading the retrieved chunks is the fastest diagnosis available. Most retrieval problems are visible in thirty seconds of looking.

Quotas worth knowing about

QUOTA	NOTE
Knowledge bases per account	Capped. Relevant if you use one per tenant
Data sources per knowledge base	Capped. Module 10's chunking is fixed per data source
numberOfResults	Maximum 100 per query
Metadata file size	10 KB per document
Ingestion job concurrency	Limits how fast a large corpus can be indexed

ON THE EXAM

The first row matters for the multi-tenancy decision: one knowledge base per tenant does not scale to thousands of tenants.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
overrideSearchType HYBRID rejected	Vector store is not OpenSearch Serverless with a filterable text field
Fewer results than numberOfRerankedResults	Capped by numberOfResults from the first stage
Reranking did not surface the right passage	It was never in the candidate set. Raise numberOfResults
Filters have no effect	The metadata was not captured at ingestion. Module 10
Tenant data leaked across customers	Relying on implicit filtering for a security boundary
Answers went stale	Data source not synced. Module 10's ingestion job
Cost rose with no more users	numberOfResults raised, or reranking enabled

ON THE EXAM

Row five is the serious one. Implicit filtering is inferred from the query, and an inference is not an access control.

Reranking reorders. Filtering restricts. numberOfResults widens. They are not interchangeable.

> An internal knowledge base serves HR, Legal and Engineering from one index. Engineers asking about parental leave receive the Legal team's policy interpretations alongside the HR policy, because both are semantically about leave.
>
> Every document already carries a department attribute in its metadata file.

What should they configure?

A Enable reranking, so the HR documents rank above the Legal ones

B Reduce numberOfResults, so fewer marginal documents are returned

C Switch overrideSearchType to HYBRID for better keyword matching

D Apply a metadata filter on the department attribute

◆ PAUSE AND DECIDE

What should they configure?

A Enable reranking, so the HR documents rank above the Legal ones

Reranking scores relevance to the query. Both document sets are genuinely relevant to 'parental leave', so it has no basis to prefer one.

B Reduce numberOfResults, so fewer marginal documents are returned

Returns fewer of everything, including the right documents. It does not distinguish between the two departments at all.

C Switch overrideSearchType to HYBRID for better keyword matching

Helps with exact identifiers. Both sets use the same vocabulary here, so keyword matching does not separate them either.

D Apply a metadata filter on the department attribute

The documents are equally relevant semantically and differ on an attribute already captured. Filtering restricts the candidate set to the right subset.

THE SIGNAL

The wrong documents are genuinely relevant. No relevance-based lever can fix that, because relevance is not what distinguishes them.

> A support knowledge base retrieves 20 chunks per query. Analysis shows the passage that actually answers the question is present in the retrieved set about 94 percent of the time, but frequently appears at position 12 or lower.
>
> The generating model only sees the top five chunks, so those answers are poor.

What should they change?

A Raise numberOfResults from 20 to 100

B Enable reranking, and return the top five after reranking

C Apply metadata filters to narrow the candidate set

D Re-chunk the corpus with a smaller chunk size

◆ PAUSE AND DECIDE

What should they change?

A Raise numberOfResults from 20 to 100

Recall is already 94 percent, so retrieval is finding it. More results worsens the ordering problem and costs more tokens.

B Enable reranking, and return the top five after reranking

Recall is good and ordering is bad. A cross-encoder scores the query and each passage together, which is exactly what first-stage search cannot do.

C Apply metadata filters to narrow the candidate set

Filtering restricts by attribute. Nothing here suggests the wrong documents are being retrieved, only that they are badly ordered.

D Re-chunk the corpus with a smaller chunk size

Expensive re-ingestion, and it does not address ordering. Module 10 also noted chunking cannot be changed on an existing data source.

THE SIGNAL

94 percent recall with poor ordering is the textbook reranking case. Recall is the first stage's job; ordering is the second stage's.

> A retailer's knowledge base is backed by Amazon S3 Vectors, chosen for cost over a large product catalogue. Users frequently search by exact SKU, and those searches return poor results.
>
> An engineer sets overrideSearchType to HYBRID to combine keyword and semantic matching. The setting is rejected.

Why, and what are the options?

A HYBRID requires reranking to be enabled first

B HYBRID is only available on managed knowledge bases

C HYBRID needs an OpenSearch Serverless store with a filterable text field. Move the store, or handle SKUs outside retrieval

D HYBRID requires numberOfResults to be 100 or fewer

◆ PAUSE AND DECIDE

Why, and what are the options?

A HYBRID requires reranking to be enabled first

They are independent settings. Reranking reorders results; search type decides how they were found.

B HYBRID is only available on managed knowledge bases

Managed KBs always use hybrid, which is different from it being exclusive to them. Customer-managed KBs can use it on a supported store.

C HYBRID needs an OpenSearch Serverless store with a filterable text field. Move the store, or handle SKUs outside retrieval

Documented: for other vector store configurations, only SEMANTIC is available. The module 09 cost decision constrained the module 26 option.

D HYBRID requires numberOfResults to be 100 or fewer

100 is the maximum for numberOfResults regardless of search type. It is unrelated to hybrid support.

THE SIGNAL

A decision made three modules ago for cost reasons removed an option here. Vector store choice and retrieval configuration are coupled.

> A compliance assistant answers questions like 'compare our data retention obligations in the UK and Germany, and say which is stricter'. It consistently answers about one jurisdiction and ignores the other.
>
> Retrieval quality for single-jurisdiction questions is excellent. Metadata, filters and reranking are all correctly configured.

What should they use?

A. AgenticRetrieveStream, which decomposes the query and retrieves iteratively

B. Raise numberOfResults so both jurisdictions fit in the candidate set

C. Switch from RetrieveAndGenerate to Retrieve, and build the prompt manually

D. Enable implicit filtering so the jurisdictions are detected automatically

◆ PAUSE AND DECIDE

What should they use?

A AgenticRetrieveStream, which decomposes the query and retrieves iteratively

A multi-part question embedded whole sits between two topics. Decomposing it into sub-queries retrieves for each, and sufficiency evaluation catches a half-answer.

B Raise numberOfResults so both jurisdictions fit in the candidate set

The query vector sits between two topics, so more results returns more of the same near-miss. The problem is the query, not the depth.

C Switch from RetrieveAndGenerate to Retrieve, and build the prompt manually

Moves work into the application without changing how the query is searched. You would end up building decomposition yourself.

D Enable implicit filtering so the jurisdictions are detected automatically

Implicit filtering narrows by inferred attributes. It would likely filter to one jurisdiction, which is the failure already occurring.

THE SIGNAL

Single-part questions work and multi-part ones fail. That pattern points at the query, not at the index or the ranking.

What to take from this module

- **Reranking reorders, filtering restricts, numberOfResults widens.** Diagnose before choosing.
- **Hybrid search needs OpenSearch Serverless** with a filterable text field. Other stores are semantic only.
- **Reranking cannot surface what was never retrieved**, and is capped by numberOfResults.
- **Explicit filters are a security boundary. Implicit filtering is not.**
- **Agentic retrieval for multi-part questions**, and GenerateQuery for 'how many'.

> ON THE EXAM
>
> Module 27 goes one step further: agents that do not only retrieve, but act.